

Лабораторная работа №36. Итоговый проект: TaskBoard

Цель работы:

- **Синтез навыков:** объединить в одном проекте ASP.NET Core, EF Core, SQLite и Fetch API.
- **Архитектурный подход:** самостоятельно построить приложение от создания репозитория до реализации CRUD-логики.
- **Закрепление Fullstack-цикла:** реализовать методы GET, POST, PUT и DELETE на стороне сервера и их вызов с фронтенда.

Краткая теория (вводная часть)

Что мы строим

TaskBoard — это интерактивная доска задач. Приложение позволит пользователю:

- Просматривать актуальный список задач.
- Добавлять новые задачи (название + описание).
- **Менять статус:** отмечать задачу как выполненную (она визуально перечёркивается).
- **Редактировать:** изменять текст задачи прямо в интерфейсе.
- **Удалять** ненужные записи.

Что изучали и что применяем сегодня

Что изучили	Где применяем
POST, DELETE, хранение данных	Контроллер TasksController
REST API, маршруты, Swagger	Все маршруты API
JSON, модели, атрибуты	Модель Task, DTO
SQLite, EF Core, миграции	AppDbContext, migrations
CRUD с базой данных	Все операции с задачами
fetch, CORS, api.js	frontend/js/api.js
PUT, редактирование, фильтрация	Редактирование задачи

Шаг 1. Создание проекта

1.1. Создание рабочей директории

1. Откройте любой терминал (например **Git Bash**).
2. Перейдите в каталог с документами.
3. Создайте директорию для лабораторной работы и папку для скриншотов:

mkdir Lab36_TaskBoard

cd Lab36_TaskBoard

mkdir img frontend

mkdir frontend/css

mkdir frontend/js

4. Создайте проект **webapi**:

dotnet new webapi -n TaskBoardApi

5. Откройте проект в VS Code:

code .

6. Откройте терминал и выполните:

cd TaskBoardApi

dotnet restore

7. Установим нужные пакеты. Выполните в терминале:

dotnet add package Swashbuckle.AspNetCore

dotnet add package Microsoft.EntityFrameworkCore

dotnet add package Microsoft.EntityFrameworkCore.Sqlite

dotnet add package Microsoft.EntityFrameworkCore.Tools

dotnet restore

8. Создайте папки:

mkdir Models Data Controllers

1.2. .editorconfig

В корне папки создайте файл **.editorconfig**. Добавьте в ваш **.editorconfig** файл следующие правила (все скобки на той же строке):

```
[*.cs]
csharp_new_line_before_open_brace = none
```

1.3. Инициализация Git

1. Выполните в терминале:

cd ..

git init -b main

cd TaskBoardApi

dotnet new gitignore

2. Выполните в терминале:

cd ..

touch README.md

git add .

git commit -m "Initial commit: TaskBoard project"

3. Создайте пустой **public-репозиторий** на GitHub с именем: **Lab36_TaskBoard**

Не добавляйте README, .gitignore, License

4. Привяжите и отправьте:

git remote add origin <URL_репозитория>

git push -u origin main

5. Сделайте **скриншот терминала** с выполненными командами (commit, remote add, push) и сохраните в папку img с именем: **gitPushLab36_FIO.png**

Шаг 2. Backend — модель и база данных

2.1. Создание модели задачи

Модель — это C#-класс, на основе которого Entity Framework создаст таблицу в базе данных.

1. В папке **Models** создайте файл **TaskItem.cs**

2. Реализуйте класс:

- Подключение атрибутов:

```
using System.ComponentModel.DataAnnotations;
```

Это позволит нам задать правила валидации данных.

- Пространство имен:

```
namespace TaskBoardApi.Models;
```

- Объявление класса TaskItem:

```
public class TaskItem { }
```

3. Добавьте свойства с атрибутами внутрь класса TaskItem:

- Id: Первичный ключ (создается автоматически)

```
public int Id { get; set; }
```

- Title: Заголовок задачи, с атрибутами [Required] (поле не может быть пустым) и [MaxLength(200)] (ограничение по длине)

```
[Required]
[MaxLength(200)]
public string Title { get; set; } = string.Empty;
```

- Description: Описание задачи. С атрибутом

```
[MaxLength(1000)]
public string Description { get; set; } = string.Empty;
```

- IsCompleted: Флаг выполнения (тип bool), по умолчанию false

```
public bool IsCompleted { get; set; } = false;
```

- CreatedAt: Дата создания, по умолчанию DateTime.UtcNow.

```
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
```

***Важное примечание:** Мы назвали класс `TaskItem`, а не просто `Task`. В языке C# имя `Task` зарезервировано для асинхронных операций. Если назвать модель так же, возникнет конфликт имен, и проект перестанет собираться.*

2.2. Создание контекста базы данных (DbContext)

DbContext — это главный класс, который отвечает за взаимодействие с базой данных (сохранение, удаление, поиск).

1. В папке **Data** создайте файл **AppDbContext.cs**.

2. Пропишите структуру по шагам:

- Подключите библиотеки: Вам нужны **Microsoft.EntityFrameworkCore** и ваша папка с моделями **TaskBoardApi.Models**.

```
using Microsoft.EntityFrameworkCore;
using TaskBoardApi.Models;

namespace TaskBoardApi.Data;
```

- Объявите класс: **AppDbContext**:

```
public class AppDbContext : DbContext { }
```

- Добавьте конструктор: Он необходим для передачи настроек (например, строки подключения) из Program.cs.

```
public class AppDbContext : DbContext {
    + public AppDbContext(DbContextOptions<AppDbContext> options)
      : base(options) { }
```

- Определите таблицу:

```
public DbSet<TaskItem> Tasks { get; set; }
```

Говорим Entity Framework, что в базе должна быть таблица *Tasks*, основанная на модели *TaskItem*.

3. Заполните базу начальными данными (Seeding): Чтобы после создания базы нам было с чем работать, переопределим метод **OnModelCreating**.

```
protected override void OnModelCreating(ModelBuilder modelBuilder) {  
    base.OnModelCreating(modelBuilder);  
}
```

4. Внутри него, используя *modelBuilder*, добавьте 4 задачи:

```
protected override void OnModelCreating(ModelBuilder modelBuilder) {  
    base.OnModelCreating(modelBuilder);  
    modelBuilder.Entity<TaskItem>().HasData(  
        new TaskItem {  
            Id = 1,  
            Title = "Изучить ASP.NET Core",  
            Description = "Контроллеры, маршруты, middleware",  
            IsCompleted = true,  
            CreatedAt = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc)  
        },
```

```
        new TaskItem {  
            Id = 2,  
            Title = "Подключить SQLite",  
            Description = "EF Core, миграции, DbContext",  
            IsCompleted = true,  
            CreatedAt = new DateTime(2026, 1, 2, 0, 0, 0, DateTimeKind.Utc)  
        },  
        new TaskItem {  
            Id = 3,  
            Title = "Написать фронтенд",  
            Description = "HTML, CSS, JavaScript, fetch",  
            IsCompleted = false,  
            CreatedAt = new DateTime(2026, 1, 3, 0, 0, 0, DateTimeKind.Utc)  
        },  
        new TaskItem {  
            Id = 4,  
            Title = "Сдать итоговый проект",  
            Description = "Показать преподавателю работающее приложение",  
            IsCompleted = false,  
            CreatedAt = new DateTime(2026, 1, 4, 0, 0, 0, DateTimeKind.Utc)  
        }  
    );  
}
```

- Задачи с ID 1 и 2 пометьте как выполненные (*IsCompleted* = true).

- Задачи с ID 3 и 4 оставьте невыполненными.
- **Важно:** Обратите внимание на `DateTimeKind.Utc` — SQLite и EF Core любят точность в форматах дат.

2.3. Program.cs

В итоговом проекте файл `Program.cs` выполняет три важные задачи: настраивает формат JSON, разрешает кросс-доменные запросы (CORS) и связывает контроллеры с базой данных.

1. Замените содержимое файла `Program.cs`.

```
using System.Text.Json;
using Microsoft.EntityFrameworkCore;
using TaskBoardApi.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers()
    .AddJsonOptions(options => {
        options.JsonSerializerOptions.PropertyNamingPolicy = JsonNamingPolicy.CamelCase;
    });

builder.Services.AddCors(options => {
    options.AddPolicy("AllowAll", policy => {
        policy.AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader();
    });
});

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite("Data Source=taskboard.db"));
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

using (var scope = app.Services.CreateScope()) {
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate();
}

if (app.Environment.IsDevelopment()) {
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseCors("AllowAll");
app.UseAuthorization();
app.MapControllers();

app.Run();
```

2. Обратите внимание на новые строки:

- **options.UseSqlite(...)** — указывает серверу использовать SQLite и называет файл базы данных taskboard.db.
- **Блок автоматической миграции:** Код using (var scope = ...) при каждом запуске проверяет базу данных. Если её нет или она устарела, он сам применит миграции. Это освобождает нас от необходимости вручную обновлять базу на других компьютерах.

2.4. Установка инструмента dotnet-ef

Если вы делаете этот шаг, до работы №31-32, то обязательно выполните и прочтите следующую инструкцию:

dotnet ef — консольный инструмент для создания миграций. Установите его глобально (один раз):

cd TaskBoardApi

dotnet tool install --global dotnet-ef

Если уже установлен — обновите:

dotnet tool update --global dotnet-ef

Проверьте установку:

dotnet ef --version

Вы должны увидеть версию:

```
└─$ dotnet ef --version
Entity Framework Core .NET Command-line Tools
10.0.3
```

Далее выполните в терминале:

dotnet restore

dotnet build

Убедитесь, что сборка прошла без ошибок.

Откройте файл **TaskBoardApi.csproj** — в нём должны появиться новые записи о пакетах:

```
<ItemGroup>
  <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="10.0.3" />
  <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.3" />
  <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="10.0.3" />
  <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.3">
    <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    <PrivateAssets>all</PrivateAssets>
  </PackageReference>
  <PackageReference Include="Swashbuckle.AspNetCore" Version="10.1.4" />
</ItemGroup>
```

ВНИМАНИЕ! У вас должны совпадать версии пакетов! В данном примере везде версия 10.0.3! Обратите на это внимание, иногда они отличались, в этом случае установите везде одинаковую версию и заново соберите проект (dotnet restore)

2.5. Создание базы данных (Миграции)

Миграции — это «чертежи», по которым Entity Framework строит таблицы в SQLite на основе ваших C#-классов.

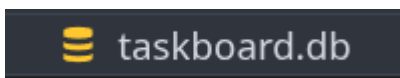
1. Убедитесь, что вы находитесь в папке проекта **TaskBoardApi**.

2. Выполните в терминале команды:

dotnet ef migrations add InitialCreate

dotnet ef database update

Результат: В корне папки TaskBoardApi должен появиться файл **taskboard.db**. Это и есть ваша база данных.



Практическое задание

dotnet build

Убедитесь что сборка прошла без ошибок.

Выполните в терминале:

git add .

git commit -m "Step 2: model, DbContext, migration"

git push

Шаг 3. Backend — контроллер (CRUD)

Контроллер TasksController будет отвечать за все операции с задачами. В отличие от предыдущей работы, здесь мы используем **инъекцию зависимостей (DI)** для доступа к базе данных и **асинхронное программирование (async/await)**, чтобы сервер не блокировался во время тяжёлых запросов к диску.

3.1. Подготовка каркаса и внедрение зависимости

1. В папке **Controllers** создайте файл **TasksController.cs**

2. **Пропишите using и namespace:** Подключите контроллеры, Entity Framework и свои папки Models и Data


```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using TaskBoardApi.Models;
using TaskBoardApi.Data;
```

3. Объявите класс с атрибутами: [ApiController] и [Route("api/[controller]")].

```
[ApiController]
[Route("api/[controller]")]
public class TasksController : ControllerBase { }
```

Далее пишем внутри класса

4. Настройте доступ к БД:

- Создайте закрытое поле _db

```
private readonly ApplicationDbContext _db;
```

- Создайте **конструктор**, который принимает ApplicationDbContext db и записывает его в поле. Теперь контроллер умеет общаться с базой

```
public TasksController(ApplicationDbContext db) {
    _db = db;
}
```

3.2. Реализация методов

- **Чтение (GET):**

- В методе GetAll() мы добавили умную сортировку: .OrderBy(t => t.IsCompleted) поднимет невыполненные задачи вверх, а .ThenByDescending(t => t.CreatedAt) покажет самые свежие задачи первыми внутри своих групп.

```
[HttpGet]
public async Task<ActionResult<List<TaskItem>>> GetAll() {
    var tasks = await _db.Tasks
        .OrderBy(t => t.IsCompleted)
        .ThenByDescending(t => t.CreatedAt)
        .ToListAsync();
    return Ok(tasks);
}
```

- Метод GetById(int id) ищет задачу по первичному ключу через FindAsync.

```
[HttpGet("{id}")]
public async Task<ActionResult<TaskItem>> GetById(int id) {
    var task = await _db.Tasks.FindAsync(id);
    if (task is null) {
        return NotFound(new { message = $"Задача с id={id} не найдена" });
    }
    return Ok(task);
}
```

- **Создание (POST):**

- При добавлении новой задачи мы принудительно обнуляем Id (чтобы база сама его выдала) и выставаем текущую дату.
- Обязательно вызывайте await _db.SaveChangesAsync(), иначе изменения не запишутся в файл .db.

```
[HttpPost]
public async Task<ActionResult<TaskItem>> Create([FromBody] TaskItem task) {
    if (string.IsNullOrEmpty(task.Title)) {
        return BadRequest(new { message = "Название задачи не может быть пустым" });
    }
    task.Id = 0;
    task.IsCompleted = false;
    task.CreatedAt = DateTime.UtcNow;
    _db.Tasks.Add(task);
    await _db.SaveChangesAsync();
    return CreatedAtAction(nameof(GetById), new { id = task.Id }, task);
}
```

- **Обновление (PUT):**

- Он находит существующую задачу по ID и перезаписывает её поля (Title, Description, IsCompleted) данными, пришедшими от фронтенда.

```
[HttpPut("{id}")]
public async Task<ActionResult<TaskItem>> Update(int id,
[FromBody] TaskItem updated) {
    var task = await _db.Tasks.FindAsync(id);
    if (task is null) {
        return NotFound(new { message = $"Задача с id={id} не найдена" });
    }
    if (string.IsNullOrEmpty(updated.Title)) {
        return BadRequest(new { message = "Название задачи не может быть пустым" });
    }
    task.Title = updated.Title;
    task.Description = updated.Description;
    task.IsCompleted = updated.IsCompleted;
    await _db.SaveChangesAsync();
    return Ok(task);
}
```

- **Удаление (DELETE):**

```
[HttpDelete("{id}")]
public async Task<ActionResult> Delete(int id) {
    var task = await _db.Tasks.FindAsync(id);
    if (task is null) {
        return NotFound(new { message = $"Задача с id={id} не найдена" });
    }
    _db.Tasks.Remove(task);
    await _db.SaveChangesAsync();
    return NoContent();
}
```

- Находит задачу и удаляет её из контекста. Не забывайте возвращать `NoContent()`, если всё прошло успешно.

Практическое задание: Тестирование API

Прежде чем браться за фронтенд, нужно убедиться, что "мотор" нашего приложения работает идеально.

1. **Запуск:** Выполните **dotnet run** и откройте Swagger **http://localhost:5000/swagger**

2. **Тест-драйв:**

- **GET:** Убедитесь, что задачи с `isCompleted: false` идут в начале списка.
- **POST:** Создайте задачу "Купить хлеб".
- **PUT:** Возьмите ID созданной задачи и через метод PUT измените ей `isCompleted` на `true`. Проверьте через GET, что она опустилась в низ списка.
- **Ошибки:** Попробуйте запросить задачу с несуществующим ID (например, 999) — вы должны получить статус **404 Not Found**.

Сделайте скриншот Swagger → **img/step3_swaggerLab36_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 3: TasksController with full CRUD"

git push

Шаг 4. Frontend — HTML и CSS

4.1. HTML

1. В папке **frontend** создайте файл **index.html**

2. Вставьте следующий код (**Обратите внимание, при вставке возможны проблемы, проверьте попробовав сохранить файл Ctrl+S и посмотрев на ошибки Prettier!**):

```
<!DOCTYPE html>
<html lang="ru">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>TaskBoard</title>
<link rel="stylesheet" href="css/style.css" />
</head>
<body>
<header>
<h1>✔ TaskBoard</h1>
<p class="subtitle">Список задач</p>
</header>
<main>
<!-- Форма добавления задачи -->
<section class="add-form">
<h2>Новая задача</h2>
<div class="form-col">
<input type="text" id="inputTitle" placeholder="Название задачи *" />
<textarea id="inputDescription" placeholder="Описание (необязательно)" rows="2"></textarea>
<button id="btnAdd">Добавить задачу</button>
</div>
<p class="error-message" id="errorMessage"></p>
</section>
<!-- Фильтры -->
<section class="filters">
<button class="filter-btn active" data-filter="all">Все</button>
<button class="filter-btn" data-filter="active">В работе</button>
<button class="filter-btn" data-filter="completed">Выполненные</button>
<span class="tasks-count" id="tasksCount"></span>
</section>
<!-- Статус загрузки -->
<p class="loading-text" id="loadingText">Загружаем задачи ... </p>
<!-- Список задач -->
<ul class="tasks-list" id="tasksList"></ul>
</main>
<script src="js/api.js"></script>
<script src="js/app.js"></script>
</body>
</html>
```

4.2. CSS

Чтобы страница выглядела современно, добавим стили.

1. В папке **frontend/css** создайте файл **style.css**
2. Используйте CSS-сетку (Flexbox) для автоматического расположения карточек.

Создайте **frontend/css/style.css** и вручную перепишите:

```
/* Общие стили */
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}
body {
font-family: Arial, sans-serif;
background: #f0f2f5;
color: #1a1a1a;
```

```
min-height: 100vh;
}
/* Шапка */
header {
background: #2d3748;
color: white;
text-align: center;
padding: 2rem;
}
header h1 {
font-size: 2rem;
margin-bottom: 0.4rem;
}
.subtitle {
color: #a0aec0;
font-size: 0.95rem;
}
/* Основной контент */
main {
max-width: 700px;
margin: 0 auto;
padding: 2rem 1rem;
}
/* Форма добавления */
.add-form {
background: white;
border-radius: 12px;
padding: 1.5rem;
margin-bottom: 1.5rem;
box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}
.add-form h2 {
font-size: 1rem;
color: #4a5568;
margin-bottom: 1rem;
}
.form-col {
display: flex;
flex-direction: column;
gap: 0.6rem;
}
.form-col input,
.form-col textarea {
padding: 0.65rem 0.9rem;
border: 1px solid #ddd;
border-radius: 8px;
font-size: 0.95rem;
font-family: inherit;
outline: none;
resize: none;
transition: border-color 0.2s;
}
.form-col input:focus,
.form-col textarea:focus {
border-color: #667eea;
}
.form-col button {
padding: 0.65rem;
background: #667eea;
color: white;
border: none;
border-radius: 8px;
font-size: 0.95rem;
cursor: pointer;
transition: background 0.2s;
}
.form-col button:hover {
background: #5a6fd6;
}
```

```
.form-col button:disabled {
background: #a0aec0;
cursor: not-allowed;
}

.error-message {
color: #e53e3e;
font-size: 0.85rem;
margin-top: 0.4rem;
min-height: 1.2rem;
}

/* Фильтры - */
.filters {
display: flex;
align-items: center;
gap: 0.5rem;
margin-bottom: 1rem;
flex-wrap: wrap;
}

.filter-btn {
padding: 0.4rem 1rem;
border: 1px solid #ddd;
border-radius: 20px;
background: white;
font-size: 0.85rem;
cursor: pointer;
transition: all 0.15s;
color: #4a5568;
}

.filter-btn:hover {
border-color: #667eea;
color: #667eea;
}

.filter-btn.active {
background: #667eea;
border-color: #667eea;
color: white;
}

.tasks-count {
margin-left: auto;
font-size: 0.85rem;
color: #718096;
}

/* Текст загрузки */
.loading-text {
text-align: center;
color: #718096;
padding: 2rem;
}

/* Список задач */
.tasks-list {
list-style: none;
display: flex;
flex-direction: column;
gap: 0.6rem;
}

/* Карточка задачи */
.task-item {
background: white;
border-radius: 10px;
padding: 1rem 1.25rem;
box-shadow: 0 1px 4px rgba(0, 0, 0, 0.07);
border-left: 4px solid #667eea;
transition: box-shadow 0.15s;
}

.task-item:hover {
box-shadow: 0 3px 10px rgba(0, 0, 0, 0.1);
}

.task-item.completed {
border-left-color: #68d391;
}
```

```
opacity: 0.7;
}
/* Режим просмотра */
.task-view {
display: flex;
align-items: flex-start;
gap: 0.75rem;
}
.task-checkbox {
width: 20px;
height: 20px;
cursor: pointer;
accent-color: #667eea;
flex-shrink: 0;
margin-top: 2px;
}
.task-content {
flex: 1;
}
.task-title {
font-size: 0.95rem;
font-weight: 600;
color: #2d3748;
margin-bottom: 0.2rem;
}
.task-item.completed .task-title {
text-decoration: line-through;
color: #a0aec0;
}
.task-description {
font-size: 0.82rem;
color: #718096;
margin-bottom: 0.3rem;
}
.task-date {
font-size: 0.75rem;
color: #cbd5e0;
}
.task-actions {
display: flex;
gap: 0.4rem;
flex-shrink: 0;
}
.btn-edit-task {
background: none;
border: 1px solid #bee3f8;
color: #3182ce;
padding: 0.25rem 0.6rem;
border-radius: 5px;
font-size: 0.78rem;
cursor: pointer;
transition: background 0.15s;
}
.btn-edit-task:hover {
background: #ebf8ff;
}
.btn-delete-task {
background: none;
border: 1px solid #fed7d7;
color: #e53e3e;
padding: 0.25rem 0.6rem;
border-radius: 5px;
font-size: 0.78rem;
cursor: pointer;
transition: background 0.15s;
}
.btn-delete-task:hover {
background: #fff5f5;
}
```

```
/* Режим редактирования */
.task-edit {
display: none;
flex-direction: column;
gap: 0.5rem;
}
.task-edit input,
.task-edit textarea {
padding: 0.5rem 0.7rem;
border: 1px solid #ddd;
border-radius: 6px;
font-size: 0.9rem;
font-family: inherit;
outline: none;
resize: none;
transition: border-color 0.2s;
}
.task-edit input:focus,
.task-edit textarea:focus {
border-color: #667eea;
}
.edit-buttons {
display: flex;
gap: 0.5rem;
}
.btn-save-task {
padding: 0.35rem 0.9rem;
background: #667eea;
color: white;
border: none;
border-radius: 6px;
font-size: 0.82rem;
cursor: pointer;
transition: background 0.15s;
}
.btn-save-task:hover {
background: #5a6fd6;
}
.btn-cancel-task {
padding: 0.35rem 0.9rem;
background: none;
border: 1px solid #ddd;
color: #718096;
border-radius: 6px;
font-size: 0.82rem;
cursor: pointer;
transition: background 0.15s;
}
.btn-cancel-task:hover {
background: #f7fafc;
}
.edit-error {
color: #e53e3e;
font-size: 0.8rem;
min-height: 1rem;
}
/* Пустое состояние */
.empty-text {
text-align: center;
color: #a0aec0;
padding: 3rem;
font-size: 0.95rem;
}
```


Практическое задание

Откройте **frontend/index.html** через **Live Server**. Должна появиться страница с формой и текстом «Загружаем задачи...». JavaScript напишем в следующем шаге.

Выполните в терминале:

git add .

git commit -m "Step 4: frontend HTML and CSS"

git push

Шаг 5. Frontend — JavaScript (Сетевой модуль)

В итоговом проекте мы придерживаемся модульного подхода. Файл `api.js` будет содержать только логику общения с сервером. Это позволит нам легко тестировать запросы отдельно от интерфейса.

5.1. Создание сервиса API

1. В папке **frontend/js** создайте файл **api.js**
2. Реализуйте функции по шагам, обращая внимание на различия в методах HTTP:

- Базовый URL:

```
const API_URL = "http://localhost:5000/api/tasks";
```

Если ваш бэкенд запустился на другом порту, исправьте его здесь.

- Метод `getAllTasks()`, для получения всех задач:

```

async function getAllTasks() {
  const response = await fetch(API_URL);
  if (!response.ok) {
    throw new Error("Не удалось загрузить задачи");
  }
  return response.json();
}

```

Использует стандартный GET. Просто ожидаем ответа и парсим JSON.

- **Метод deleteTask(id)**, для удаления задачи:

```

async function deleteTask(id) {
  const response = await fetch(`${API_URL}/${id}`, {
    method: "DELETE",
  });
  if (!response.ok) {
    const error = await response.json();
    throw new Error(error.message || "Не удалось удалить задачу");
  }
}

```

Отправляет DELETE-запрос. Здесь тело не требуется, только ID в адресе.

- **Метод createTask(title, description)**, для создания задачи:

```

async function createTask(title, description) {
  const response = await fetch(API_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, description }),
  });
  if (!response.ok) {
    const error = await response.json();
    throw new Error(error.message || "Не удалось создать задачу");
  }
  return response.json();
}

```

Отправляет POST-запрос. Мы передаем объект с заголовком Content-Type, чтобы сервер понял формат данных.

- **Метод updateTask(id, title, description, isCompleted)**, для обновления задач:

```

async function updateTask(id, title, description, isCompleted) {
  const response = await fetch(`${API_URL}/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, description, isCompleted }),
  });
  if (!response.ok) {
    const error = await response.json();
    throw new Error(error.message || "Не удалось обновить задачу");
  }
  return response.json();
}

```

Использует **PUT**.

Обратите внимание: мы передаем id в строке адреса (URL), а остальные данные — в теле (body). Это стандарт для обновления ресурсов.

Разбор ключевых моментов

- **Шаблонные строки:** Обратите внимание на использование обратных кавычек в `${API_URL}/${id}`. Это позволяет динамически подставлять ID конкретной задачи в путь запроса.
- **Обработка ошибок:** В каждом методе мы проверяем `!response.ok`. Если сервер вернул ошибку (например, 400 или 404), мы пытаемся прочитать сообщение от сервера (`error.message`) и пробросить его дальше. Это поможет нам вывести понятную ошибку пользователю в интерфейсе.
- **Асинхронность:** Все функции помечены как `async`, так как сетевые запросы не происходят мгновенно. Мы используем `await`, чтобы код "ждал" ответа от сервера, не замораживая при этом страницу.

5.2. Frontend — Логика приложения (app.js)

Создайте **frontend/js/app.js** и скопируйте код:

```

// Элементы страницы
const tasksList = document.getElementById("tasksList");
const loadingText = document.getElementById("loadingText");
const errorMessage = document.getElementById("errorMessage");
const inputTitle = document.getElementById("inputTitle");
const inputDesc = document.getElementById("inputDescription");
const btnAdd = document.getElementById("btnAdd");
const tasksCount = document.getElementById("tasksCount");

// Состояние
let allTasks = []; // все задачи с сервера
let activeFilter = "all"; // текущий фильтр

// Отрисовка задач

function renderTasks(tasks) {
  loadingText.style.display = "none";

  // Обновляем счётчик

```

```

const total = allTasks.length;
const completed = allTasks.filter((t) => t.isCompleted).length;
tasksCount.textContent = `${completed} из ${total} выполнено`;

if (tasks.length === 0) {
tasksList.innerHTML = '<p class="empty-text">Задач не найдено</p>';
return;
}

tasksList.innerHTML = tasks.map((task) => createTaskHTML(task)).join("");

function createTaskHTML(task) {
const date = new Date(task.createdAt).toLocaleDateString("ru-RU");
const desc = task.description ? `<p class="task-description">${escapeHtml(task.description)}</p>` : "";

return `
<li class="task-item ${task.isCompleted ? "completed" : ""}"
id="task-${task.id}">

<!-- Режим просмотра -->
<div class="task-view">
<input type="checkbox"
class="task-checkbox"
${task.isCompleted ? "checked" : ""}
onchange="handleToggle(${task.id}, this.checked)" />
<div class="task-content">
<p class="task-title">${escapeHtml(task.title)}</p>
${desc}
<p class="task-date">Создано: ${date}</p>
</div>
<div class="task-actions">
<button class="btn-edit-task"
onclick="showEditMode(${task.id})">

</button>
<button class="btn-delete-task"
onclick="handleDelete(${task.id})">

</button>
</div>
</div>

<!-- Режим редактирования -->
<div class="task-edit" id="edit-${task.id}">
<input type="text"
id="editTitle-${task.id}"
value="${escapeHtml(task.title)}" />
<textarea id="editDesc-${task.id}"
rows="2">${escapeHtml(task.description)}</textarea>
<p class="edit-error" id="editError-${task.id}"></p>
<div class="edit-buttons">
<button class="btn-save-task"
onclick="handleUpdate(${task.id}, ${task.isCompleted})">
 Сохранить
</button>
<button class="btn-cancel-task"
onclick="hideEditMode(${task.id})">
Отмена
</button>
</div>
</div>

</li>
`;
}

// Переключение режимов карточки

function showEditMode(id) {
const item = document.getElementById(`task-${id}`);
const viewMode = item.querySelector(".task-view");
const editMode = document.getElementById(`edit-${id}`);

viewMode.style.display = "none";
editMode.style.display = "flex";

// Фокус на поле заголовка
document.getElementById(`editTitle-${id}`).focus();

```

```

}

function hideEditMode(id) {
const item = document.getElementById(`task-${id}`);
const viewMode = item.querySelector(".task-view");
const editMode = document.getElementById(`edit-${id}`);

viewMode.style.display = "flex";
editMode.style.display = "none";
}

// Загрузка задач

async function loadTasks() {
try {
allTasks = await getAllTasks();
applyFilter();
} catch (error) {
loadingText.style.display = "none";
tasksList.innerHTML = `

❌ Ошибка: ${error.message}</p>`;
}
}

// Фильтрация

function applyFilter() {
let filtered = allTasks;

if (activeFilter === "active") {
filtered = allTasks.filter((t) => !t.isCompleted);
} else if (activeFilter === "completed") {
filtered = allTasks.filter((t) => t.isCompleted);
}

renderTasks(filtered);
}

// Добавление задачи

async function handleAdd() {
const title = inputTitle.value.trim();
const desc = inputDesc.value.trim();

errorMessage.textContent = "";

if (!title) {
errorMessage.textContent = "Введите название задачи";
inputTitle.focus();
return;
}

btnAdd.disabled = true;
btnAdd.textContent = "Добавляем ... ";

try {
await createTask(title, desc);
inputTitle.value = "";
inputDesc.value = "";
await loadTasks();
} catch (error) {
errorMessage.textContent = error.message;
} finally {
btnAdd.disabled = false;
btnAdd.textContent = "Добавить задачу";
}
}

// Отметить выполненной / невыполненной

async function handleToggle(id, isCompleted) {
// Находим задачу в локальном массиве
const task = allTasks.find((t) => t.id === id);
if (!task) return;

try {
await updateTask(id, task.title, task.description, isCompleted);
await loadTasks();
} catch (error) {
alert("Ошибка: " + error.message);
}
}


```

```

// Откатываем чекбокс обратно
await loadTasks();
}
}

// Редактирование задачи

async function handleUpdate(id, isCompleted) {
const title = document.getElementById(`editTitle-${id}`).value.trim();
const desc = document.getElementById(`editDesc-${id}`).value.trim();
const errorEl = document.getElementById(`editError-${id}`);

errorEl.textContent = "";

if (!title) {
errorEl.textContent = "Название не может быть пустым";
return;
}

try {
await updateTask(id, title, desc, isCompleted);
await loadTasks();
} catch (error) {
errorEl.textContent = error.message;
}
}

// Удаление задачи
async function handleDelete(id) {
const task = allTasks.find((t) => t.id === id);
const name = task ? `«${task.title}»` : `#${id}`;

if (!confirm(`Удалить задачу ${name}?`)) return;

try {
await deleteTask(id);
await loadTasks();
} catch (error) {
alert("Ошибка при удалении: " + error.message);
}
}

// Обработчики событий
btnAdd.addEventListener("click", handleAdd);

inputTitle.addEventListener("keydown", function (e) {
if (e.key === "Enter") handleAdd();
});

// Кнопки фильтров
document.querySelectorAll(".filter-btn").forEach((btn) => {
btn.addEventListener("click", function () {
// Убираем active у всех кнопок
document.querySelectorAll(".filter-btn").forEach((b) => b.classList.remove("active"));

// Добавляем active к нажатой
this.classList.add("active");

activeFilter = this.dataset.filter;
applyFilter();
});
});

// Вспомогательные функции
function escapeHtml(str) {
return String(str ?? "")
.replace(/&/g, "&amp;")
.replace(/</g, "&lt;")
.replace(/>/g, "&gt;")
.replace(/"/g, "&quot;");
}

// Запуск
loadTasks();

```

Этот файл — «мозг» вашего интерфейса. Он не только отправляет запросы к API, но и управляет тем, что пользователь видит на экране в конкретный момент: режим редактирования, фильтры или счетчик задач.

Основные концепции кода:

1. Глобальное состояние (State):

- **allTasks**: мы храним копию всех задач с сервера в оперативной памяти браузера. Это позволяет быстро фильтровать их без лишних запросов к БД.
- **activeFilter**: переменная, которая помнит, какую вкладку выбрал пользователь («Все», «В работе» или «Выполненные»).

2. Динамическое переключение режимов:

- Обратите внимание на функции `showEditMode` и `hideEditMode`. В каждой карточке задачи у нас есть два блока: `.task-view` (просмотр) и `.task-edit` (форма редактирования). JavaScript переключает их видимость (`display: none/flex`), имитируя переход на другую страницу.

3. Безопасность (`escapeHtml`):

- **Важный момент!** В коде используется функция `escapeHtml`. Она заменяет спецсимволы (вроде `<` или `>`) на безопасные коды. Это защищает ваше приложение от XSS-атак (когда вместо названия задачи кто-то может попытаться вставить вредоносный скрипт).

4. События и фильтрация:

- Мы используем `dataset.filter` у кнопок, чтобы понимать, какой фильтр применить. Метод `filter()` создает новый массив на основе условий, который затем отрисовывается функцией `renderTasks`.

Разбор ключевых функций:

- **handleToggle**: Срабатывает при клике на чекбокс. Она берет текущие данные задачи из массива `allTasks`, меняет только статус и отправляет PUT-запрос на сервер.
- **handleUpdate**: Считывает данные из полей ввода внутри карточки и сохраняет изменения.
- **Счетчик задач**: В функции `renderTasks` мы вычисляем количество выполненных задач "на лету", используя шаблонную строку: ``${completed} из ${total} выполнено``.

Практическое задание

1. Убедитесь что API запущен (**dotnet run** в папке TaskBoardApi)
2. Откройте **frontend/index.html** через **Live Server**
3. Проверьте полный цикл:
 - Загрузились 4 начальные задачи, две выполнены
 - Добавьте новую задачу с описанием
 - Отметьте задачу выполненной через чекбокс — она перечёркивается
 - Нажмите «» — карточка переключается в режим редактирования
 - Измените название, нажмите « Сохранить»
 - Нажмите фильтр «В работе» — видны только невыполненные
 - Нажмите фильтр «Выполненные» — видны только выполненные
 - Удалите задачу с подтверждением
4. Остановите сервер и запустите снова — все изменения сохранились
5. Откройте **DevTools** → **Network** — найдите **PUT-запрос** при отметке задачи

Сделайте скриншот готового приложения → [img/step5_appLab36_FIO.png](#)

Выполните в терминале:

git add .

git commit -m "Step 5: frontend JS complete - full CRUD working"

git push

Самостоятельное задание README.md

Создайте в корне проекта README.md. Опишите в нём:

1. Кто выполнил работу (ФИО, группа).
2. Описание проекта — что умеет TaskBoard
3. Краткую инструкцию: как запустить бэкенд (dotnet run) и как открыть фронтенд.
4. Главные выводы — минимум 5 пунктов своими словами

Выполните в терминале:

git add .

git commit -m "Lab36 complete: TaskBoard with all features"

git push

Итоговая таблица: что применили в лабораторной

Концепция	Описание
dotnet new webapi	Создание проекта Web API
dotnet add package	Установка NuGet-пакетов
TaskItem вместо Task	Избегаем конфликта с встроенным классом C#
[Required] / [MaxLength]	Валидация модели
AppDbContext	Класс для работы с базой данных
HasData(...)	Начальные данные в миграции
db.Database.Migrate()	Автоматические миграции при запуске
OrderBy + ThenByDescending	Сортировка по двум полям
FindAsync /ToListAsync	Асинхронные операции с БД
task.Id = 0	EF Core сам присвоит Id
GET / POST / PUT / DELETE	Полный CRUD через HTTP-методы
api.js — отдельный модуль	Все fetch-запросы в одном файле
allTasks — локальный массив	Фильтрация без лишних запросов к серверу
activeFilter — переменная состояния	Текущий выбранный фильтр
applyFilter()	Применяет фильтр к локальному массиву
escapeHtml()	Защита от XSS при вставке в innerHTML
task-view / task-edit	Два режима карточки
data-filter атрибут	Хранение значения фильтра в HTML-кнопке
dataset.filter	Чтение data-* атрибута в JavaScript
querySelectorAll + forEach	Навешивание событий на группу кнопок